

Práctica # 1: PROGRAMACIÓN EN RADIO DEFINIDA POR SOFTWARE (GNURADIO)

Diego Steven Rincón Rodríguez - 2204212
Cesar Augusto Veloza Rodríguez - 2214060
Javier Esteban Torres Lucero - 2220405

https://github.com/EstebanToLu/CommII_A1_A1H

Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones
Universidad Industrial de Santander

1 de septiembre de 2026

Abstract—This paper presents the design, implementation, and experimental validation of three custom signal processing blocks developed in Python within the GNU Radio Software-Defined Radio (SDR) framework: a discrete Accumulator, a Differentiator, and a real-time Statistical Analyzer. The Accumulator was evaluated using discrete signals to demonstrate its recursive, state-retaining behavior ($y[n] = y[n-1] + x[n]$). The Differentiator was tested with a 500 Hz unipolar triangular wave ($A = 1$, $f_s = 32$ kHz), successfully generating a square wave output whose step magnitude (≈ 0.0325) matched the theoretical derivative within a 5 % error margin. The Statistical Analyzer computes online running metrics—including mean, mean square, RMS, average power, and standard deviation—updated continuously over the complete sample history. Internal consistency of these real-time calculations was validated against fundamental theoretical identities ($RMS = \sqrt{\overline{x^2}}$ and $\overline{x^2} = \sigma_x^2 + \bar{x}^2$). Furthermore, by evaluating a sawtooth signal under varying levels of additive Gaussian noise (0, 0.05, and 0.15), the statistical analyzer demonstrated predictable variance escalation ($\sigma_{total}^2 \approx \sigma_{signal}^2 + \sigma_{noise}^2$ with 2–3 % error), proving its effectiveness for online noise characterization and channel quality assessment without requiring a clean reference signal.

Index Terms—Software-Defined Radio (SDR), GNU Radio, Embedded Python Block, Discrete Accumulator, Differentiator, Online Statistical Analysis, Noise Characterization.

I Introducción

La Radio Definida por Software (SDR por sus siglas en inglés) constituye una de las tendencias más relevantes en el diseño moderno de sistemas de comunicaciones, al trasladar hacia el dominio del software etapas de procesamiento que tradicionalmente se implementaban en hardware dedicado. Esta flexibilidad permite modificar, depurar y extender el comportamiento de un sistema sin necesidad de re diseñar circuitos físicos, lo cual

resulta especialmente valioso en un contexto académico donde el objetivo es comprender a fondo los algoritmos que subyacen a cada etapa de procesamiento de una señal. En esta practica se desarrollaran tres bloques

programables en Python: un **Acumulador**, que actúa como integrador discreto; un **Diferenciador**, que resalta las variaciones rápidas de una señal; y un **Analizador estadístico**, capaz de calcular en tiempo real métricas como la media, el valor RMS, la potencia promedio, la media cuadrática y la desviación estándar de una señal de entrada. A lo largo del informe se documenta el diseño la implementación y la validación de cada uno de los bloques que mencionados, incluyendo el hallazgo y la corrección de un error de implementación detectado en el bloque estadístico, así como una aplicación de las métricas estadísticas para caracterizar el efecto del ruido sobre una señal de prueba.

II Metodología

II-A Diseño e Implementación de los Bloques

Sobre GNU Radio se emplearon *Embedded Python Blocks* para implementar 3 algoritmos de procesamiento de señales en tiempo discreto

- **Acumulador(Integrador)**: calcula la suma acumulada de las muestras de entrada

$$y[n] = \sum_{k=0}^n x[k] \quad (1)$$

- **Diferenciador**: resalta las variaciones rápidas de la señal mediante la diferencia entre muestras consecutivas

$$y[n] = x[n] - x[n-1] \quad (2)$$

- **Analizador Estadístico** : calcula en tiempo real de forma acumulativa(*Running/online*, la media,

la media cuadrática, el valor RMS, la potencia promedio y la desviación estándar de la señal de entrada actualizando cada métrica con base en el historial completo de muestras procesadas y no únicamente sobre el bloque de muestras vigentes en cada llamado a `work()`.

II-B Señales de prueba y procedimiento de validación

para validar el comportamiento de cada bloque se emplearon distintas señales de prueba generadas mediante el bloque *Vector Source* de GNU radio, junto con un bloque *Throttle* para regular la velocidad de procesamiento y bloques *QT GUI Sink* para la visualización de resultados:

- El **Acumulador** se evaluó con una señal **Senoidal** ya que la integral discreta de un seno se aproxima a un coseno invertido y escalado, lo cual permite verificar tanto la amplitud como la fase de la salida contra la teoría
- El **Diferenciador** se evaluó con una señal **Triangular**, cuya derivada teórica es una onda cuadrada de magnitud constante por tramos, un resultado directo de verificar cuantitativamente
- **Bloque Estadístico** se evaluó primero con una validación cruzada entre sus propias métricas (relación $RMS = \sqrt{\overline{x^2}}$ y $\overline{x^2} = \sigma_x^2 + \bar{x}^2$), y posteriormente con una señal **diente de sierra** sumada a **ruido gaussiano** de amplitud variable, con el fin de evaluar la sensibilidad de las métricas estadísticas ante señales estocásticas y de emplear dichas métricas como herramienta para caracterizar el nivel de ruido presente en una señal.

III RESULTADOS Y ANÁLISIS DE DATOS

III-A Bloque Acumulador

Para analizar el comportamiento del bloque acumulador en GNU Radio, se implementó un diagrama de flujo compuesto por un bloque *Vector Source*, un bloque *Throttle*, un bloque acumulador denominado `e_Acum` y dos bloques *QT GUI Time Sink* para visualizar simultáneamente las señales de entrada y salida.

La señal de entrada se generó mediante el bloque *Vector Source*, utilizando un conjunto de muestras con valores positivos y negativos distribuidos aproximadamente entre -1 y 1 . Se habilitó la opción *Repeat*, permitiendo que el vector se reprodujera continuamente durante la ejecución.

La frecuencia de muestreo se estableció en:

$$f_s = 256 \text{ kHz} \quad (3)$$

El bloque *Throttle* se configuró con la misma frecuencia de muestreo, con el objetivo de limitar la velocidad de procesamiento de GNU Radio y evitar un consumo innecesario de recursos del procesador.

Posteriormente, la señal se aplicó al bloque acumulador `e_Acum`, cuyo funcionamiento corresponde a:

$$y[n] = \sum_{k=0}^n x[k] \quad (4)$$

De manera equivalente, puede expresarse de forma recursiva como:

$$y[n] = y[n-1] + x[n] \quad (5)$$

Es decir, cada muestra de salida corresponde a la suma de la muestra actual con todas las muestras procesadas anteriormente.

Para observar el comportamiento del sistema se utilizaron dos bloques *QT GUI Time Sink*. El primero permitió visualizar aproximadamente 200 muestras de la señal de entrada, mientras que el segundo se configuró con aproximadamente 2000 muestras, permitiendo observar con mayor claridad la evolución temporal de la señal acumulada.

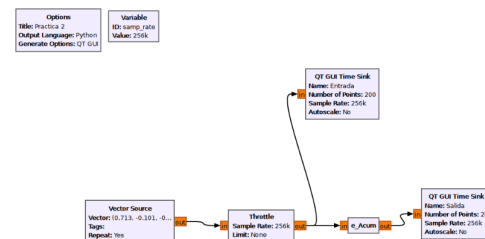


Figura 1: Acumulador

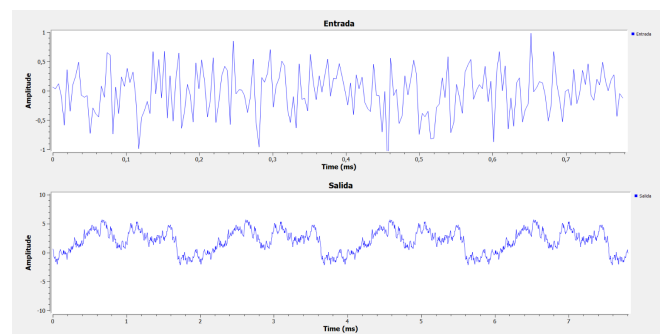


Figura 2: Señal De acumulador

En la gráfica de entrada se observa una señal discreta con variaciones aparentemente aleatorias entre valores

positivos y negativos. La amplitud se mantiene aproximadamente dentro del intervalo:

$$-1 \leq x[n] \leq 1 \quad (6)$$

La señal presenta cambios rápidos entre muestras consecutivas y no muestra una tendencia creciente o decreciente definida.

Al aplicar esta señal al bloque acumulador, la señal de salida cambia significativamente. Mientras la entrada presenta variaciones rápidas alrededor de cero, la salida presenta un comportamiento más lento y acumulativo, alcanzando aproximadamente valores entre -2 y 6 en la ventana observada.

Este comportamiento se debe a que el acumulador no procesa cada muestra de forma independiente, sino que conserva el resultado anterior y suma la nueva muestra, de acuerdo con:

$$y[n] = y[n-1] + x[n] \quad (7)$$

De esta manera, las muestras positivas incrementan el valor acumulado, mientras que las muestras negativas hacen que este disminuya.

En la gráfica obtenida se observa precisamente este fenómeno. Los pequeños cambios positivos y negativos de la señal de entrada se van sumando y producen una salida con variaciones de mayor duración. Por esta razón, la señal de salida adquiere una apariencia similar a una caminata acumulativa.

III-B Bloque Diferenciador

Se evaluó el diferenciador con una entrada triangular ($f = 500$ Hz, $A = 1$, $f_s = 32$ kHz), en lugar de la entrada tipo rampa habitual, ya que la derivada teórica de un triángulo simétrico es una onda cuadrada — un resultado más directo de verificar.

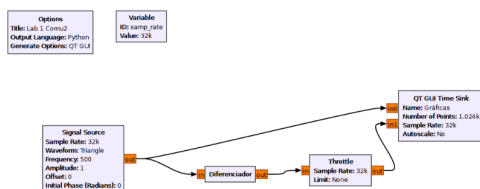


Figura 3: Bloque Diferenciador

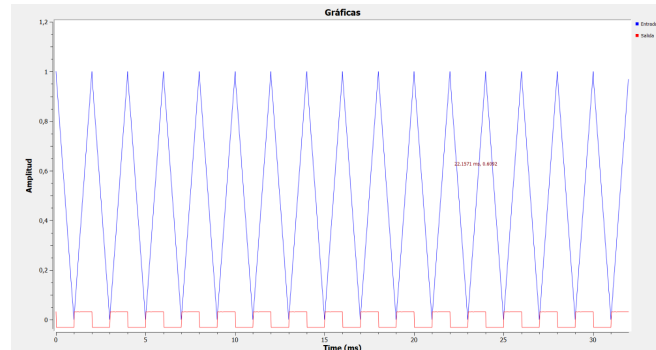


Figura 4: Señal triangular

Como se observa en la Fig. 4, la salida toma la forma de una onda cuadrada: un nivel positivo constante mientras la entrada sube, y un nivel negativo constante mientras baja, confirmando que la derivada de un triángulo es una señal cuadrada. **Verificación cuantitativa.**

Se observó que el generador de onda triangular de GNU Radio en esta configuración es unipolar (recorre de 0 a A , no de $-A$ a A), lo cual se confirmó revisando el rango del eje vertical de la señal de entrada. Para una triangular unipolar, la magnitud teórica del escalón de salida es.

$$\Delta y = \frac{2 A f}{f_s} = \frac{2 \cdot 1 \cdot 500}{32000} \approx 0,03125 \quad (8)$$

Midiendo directamente sobre la gráfica con el cursor se obtuvo $+0,0325$ en el tramo de subida y $-0,0330$ en el tramo de bajada, valores prácticamente simétricos y consistentes (diferencia $< 5\%$) con el valor teórico calculado, confirmando tanto la magnitud como la simetría esperada de la derivada.

III-C Análisis estadístico en tiempo real

Se implementó el bloque `Promedios_de_tiempos` para calcular, de forma acumulativa, la media, la media cuadrática, el RMS y la desviación estándar de una señal de entrada. Para validar la consistencia del bloque se contrastaron los valores obtenidos frente a dos relaciones teóricas independientes: $\text{RMS} = \sqrt{x^2}$ y $\bar{x}^2 = \sigma_x^2 + \bar{x}^2$.

Al ejecutar el bloque sobre una señal de prueba se obtuvieron los valores de la Tabla 1.

Se observó que se cumple la relación teórica $\text{RMS} = \sqrt{x^2}$:

$$\sqrt{2,000425} = 1,414364 \quad (9)$$

valor que coincide exactamente con el RMS reportado por el bloque.

